



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

REAL-TIME BIG DATA INGESTION PIPELINE FROM DISTRIBUTED SOURCES

A CAPSTONE PROJECT REPORT

Submitted in partial fulfilment of the requirements for the Course of

CSA1522-- CLOUD COMPUTING AND BIG DATA ANALYTICS

to the award of the degree of BACHELOR OF ENGINEERING IN AIML

Submitted by

Gali Venkata Niranjana Reddy (192525199)

Under the Supervision of

Dr. Rajaram P

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai -- 602105

SEPTEMBER 2026

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical Sciences
Chennai--602105**

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Real-Time Big Data Ingestion Pipeline from Distributed Sources" has been carried out by G. Venkata Niranjan Reddy [192525199] under the supervision of Dr. Rajaram P and is submitted in partial fulfilment of the requirements for the current semester of the B.E. Computer Science and Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

DR. A. Gnana Soundari
Professor
CSE/Machine Learning
SIMATS Engineering, Chennai -- 602105.

COURSE FACULTY

Dr Rajaram P
Professor
CSE/Data Vista
SIMATS , Chennai -- 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai--602105

DECLARATION BY THE CANDIDATE

I, G. Venkata Niranjan Reddy [192525199] of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Real-Time Big Data Ingestion Pipeline from Distributed Sources" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 5/09/2026

Signature of the Candidate

Gali Venkata Niranjan Reddy (192525199)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai--602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
M.Sandeep 192525095	Project architecture, end-to-end integration, core pipeline workflow, backend/stream-processing coordination, schema-registry integration, security hardening, deployment verification.	Designed end-to-end architecture; integrated distributed source connectors, Kafka cluster, Spark Structured Streaming jobs and sink layer; implemented schema registry and dead-letter-queue handling.	Performed end-to-end functional verification, throughput/latency benchmarking, fault-injection (broker/consumer failure) testing, and debugging of integration failures.	Prepared the individual contribution report, architecture explanation, testing tables, risk/security discussion, reflection and final documentation.	60%
G. Venkata Niranjan Reddy 192525199	Supporting implementation and module development within the team project.	Contributed to assigned producer/connector modules and feature implementation under the common project architecture.	Supported module-level testing and validation.	Contributed technical notes and project documentation.	20%
Jaya Krishna M 192524500	Supporting implementation, validation and project activities within the team.	Contributed to assigned modules and integration activities.	Supported functional testing and demonstration preparation.	Contributed technical documentation and review.	20%
Total					100%

ABSTRACT

Modern organizations generate continuous streams of data from geographically distributed and heterogeneous sources such as IoT sensors, relational databases, mobile applications, web-server logs and third-party APIs. Conventional batch-oriented extract-transform-load (ETL) workflows introduce latency between data generation and data availability, which limits the usefulness of the data for time-sensitive decisions such as fraud detection, operational monitoring and personalization. This project designs and implements a Real-Time Big Data Ingestion Pipeline that continuously captures, validates, transforms and delivers data from multiple distributed sources to downstream storage and analytics systems with minimal latency.

The proposed architecture uses Apache Kafka as a distributed, partition-based messaging backbone that decouples data producers from data consumers, Change Data Capture (CDC) connectors to stream incremental updates from relational databases, and a stream-processing layer built on Apache Spark Structured Streaming for schema validation, enrichment, deduplication and windowed aggregation. Processed records are persisted to a data lake for long-term analytics and to a low-latency serving store for near-real-time dashboards. The pipeline incorporates a schema registry to enforce data contracts, dead-letter queues for malformed records, checkpointing for exactly-once processing semantics, and horizontal partitioning for scalability.

The prototype was evaluated for ingestion throughput, end-to-end latency, fault tolerance under simulated broker and consumer failure, and schema-evolution handling. The results show that the architecture sustains high-throughput ingestion while keeping end-to-end latency within a few seconds and recovering automatically from simulated node failures without data loss. The project remains a prototype and does not claim production-scale, multi-datacenter cluster benchmarks; its principal contribution is a reusable, fault-tolerant reference architecture that connects distributed data sources, a durable messaging layer, stream processing and storage into a single coherent real-time ingestion pipeline.

Keywords: Real-Time Data Ingestion, Distributed Systems, Apache Kafka, Stream Processing, Change Data Capture, Big Data Pipeline, Fault Tolerance, Data Lake

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS / SYMBOLS	viii
1	CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	1
1.1	Background	1
1.2	Need for the Project	1
1.3	Problem Statement	2
1.4	Project Aim	2
1.5	Project Objectives	2
1.6	Scope	3
1.7	Expected Engineering Outcome	3
2	CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	4
2.1	Review Method	4
2.2	Review of Existing Work	4
2.3	Comparative Analysis	5
2.4	Research / Engineering Gap	6
2.5	Proposed Contribution	6
3	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	7
3.1	Requirements	7
3.2	Constraints & Applicable Standards	8
3.3	Design Specifications	8
3.4	Alternative Solutions & Evaluation	9
3.5	Selected Design & Detailed Design	9

Sl. No.	Title	Page No.
3.6	Trade-off Analysis	10
3.7	Sustainability, Ethics, Safety & Security	11
3.8	Risk Register	11
4	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	13
4.1	Development Approach & Resources	13
4.2	Hardware / Software Implementation & Modern Tools Used	13
4.3	Test Plan & Verification	14
4.4	Results	15
4.5	Data Analysis & Engineering Judgment	16
4.6	Comparison with Existing Solutions & Achievement of Objectives	16
4.7	Strengths & Limitations	17
4.8	Project Planning, Roles & Budget	17
5	CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION	19
5.1	Conclusion	19
5.2	Future Work	19
5.3	Professional Learning & Individual Reflection	20
	REFERENCES	22
	APPENDICES	23

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	High-level architecture of the real-time ingestion pipeline	1
3.1	Data flow from distributed sources to the serving layer	9
4.1	Kafka topic-partition layout used in the prototype	13
4.2	Throughput vs. latency test results	16

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative Analysis	5
3.1	Stakeholder Requirements	7
3.2	Functional and Non-Functional Requirements	8
3.3	Constraints & Applicable Standards	8
3.4	Alternative Solutions & Evaluation	9
3.5	Trade-off Analysis	10
3.6	Risk Register	11
4.1	Implementation Tools and Resources	13
4.2	Test Plan and Verification	14
4.3	Prototype Test Results	15
4.4	Comparison with Existing Solutions & Achievement of Objectives	16
4.5	Project Planning and Individual Roles	17
4.6	Budget Summary	18

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description
API	Application Programming Interface
CDC	Change Data Capture
CAP	Consistency, Availability, Partition tolerance (theorem)
DLQ	Dead-Letter Queue
ETL	Extract, Transform, Load
HDFS	Hadoop Distributed File System
IoT	Internet of Things
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
ms	Millisecond
PII	Personally Identifiable Information
QoS	Quality of Service
REST	Representational State Transfer
RPO / RTO	Recovery Point Objective / Recovery Time Objective
SLA	Service Level Agreement
TLS	Transport Layer Security
YARN	Yet Another Resource Negotiator
ZooKeeper	Distributed coordination service used by Kafka clusters

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Enterprises today operate in an environment where data is produced continuously and simultaneously by many independent, geographically distributed systems: IoT sensors on factory floors, mobile applications used by millions of customers, relational databases that record every transaction, application and infrastructure logs, and third-party APIs. Each of these sources produces data at a different rate, in a different format, and with different reliability guarantees.

Traditional data-integration approaches rely on scheduled batch ETL jobs that periodically extract data from source systems, transform it and load it into a warehouse. This pattern was adequate when analytics were performed once a day, but it introduces latency of minutes to hours between an event occurring and that event becoming visible to downstream consumers. This latency is unacceptable for use cases such as fraud detection, operational monitoring, real-time personalization and IoT-based alerting, where the value of data decays rapidly with time.

Stream-processing research and industry practice have converged on architectures in which a durable, partitioned commit log acts as the central nervous system of the data platform, decoupling producers from consumers and allowing multiple downstream systems to consume the same stream of events independently. This project adopts that principle and builds a Real-Time Big Data Ingestion Pipeline that continuously senses, transports, validates and delivers data from distributed sources to storage and analytics systems, replacing the traditional periodic batch cycle with a continuous flow.

1.2 Need for the Project

- Data relevant to a single business question is frequently scattered across sensors, databases, logs and external APIs that were never designed to work together.
- Batch ETL pipelines introduce hours of latency between data generation and data availability, which delays time-sensitive decisions.
- Manually reconciling data extracted from multiple heterogeneous sources is error-prone and does not scale as the number of sources grows.
- Downstream analytics, dashboards and alerting systems increasingly require freshness measured in seconds rather than hours.
- Uncontrolled schema changes at a source system can silently break brittle point-to-point integrations.
- Scaling ingestion horizontally, and recovering automatically from node or network failure, is difficult to achieve with a single ad-hoc consumer script.

1.3 Problem Statement

Design and implement a prototype real-time data-ingestion pipeline that continuously collects data from multiple distributed and heterogeneous sources, guarantees reliable, ordered and fault-tolerant delivery, applies schema validation and light transformation, and makes the data available to downstream consumers within a bounded latency budget, under realistic constraints of network variability, source schema evolution and limited prototype-scale infrastructure.

The problem is non-trivial because the pipeline must simultaneously satisfy high throughput, low latency, exactly-once (or effectively-once) delivery semantics, horizontal scalability and observability, while remaining tolerant to the failure of any individual broker, connector or processing node.

1.4 Project Aim

To design and develop a scalable, fault-tolerant Real-Time Big Data Ingestion Pipeline that integrates heterogeneous, distributed data sources through a durable messaging backbone and a stream-processing layer, enabling near-real-time analytics and monitoring while preserving data integrity and supporting schema evolution.

1.5 Project Objectives

1. Design an end-to-end architecture connecting distributed sources, an ingestion layer, a stream-processing layer and a storage/serving layer.
2. Implement producers and connectors that capture data from simulated IoT devices, relational-database change events (via Change Data Capture) and application-log streams.
3. Configure a partitioned, replicated Apache Kafka cluster as the central messaging backbone for decoupled and durable delivery.
4. Build a stream-processing layer using Apache Spark Structured Streaming for schema validation, deduplication, enrichment and windowed aggregation.
5. Persist processed data to a data lake for batch analytics and to a low-latency serving store for near-real-time dashboards.
6. Implement a schema registry and a dead-letter-queue mechanism to handle malformed records and evolving schemas.
7. Ensure fault tolerance through replication, checkpointing and automatic consumer-group rebalancing.
8. Validate throughput, end-to-end latency and failure-recovery behaviour through controlled prototype test scenarios.
9. Document the engineering trade-offs, risks, limitations and individual contribution.

1.6 Scope

Included scope: prototype data generation from simulated IoT sensors, a relational database instrumented with a CDC connector, and application-log producers; Kafka-based ingestion

with topic partitioning and replication; Spark Structured Streaming transformations; dual sinks comprising a Parquet-based data lake and a low-latency serving store; a schema registry; and a monitoring dashboard.

Excluded scope: production, multi-datacenter cluster deployment; contractual enterprise service-level agreements; integration with proprietary paid third-party data platforms; and a fully validated large-scale machine-learning model trained on the ingested data. Assumptions and boundaries include a prototype-scale cluster running on commodity hardware or a cloud free tier, controlled test-data volumes, network conditions simulated for failure testing, and human oversight for approving breaking schema changes.

1.7 Expected Engineering Outcome

The intended outcome is a working prototype that demonstrates a complete data-to-insight chain: source event generation -> producer serialization -> Kafka topic ingestion -> stream processing (validation, enrichment, deduplication, windowed aggregation) -> dual sink (data lake and serving store) -> dashboard visualization, together with fault-injection tests that show automatic recovery without data loss.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review was conducted using peer-reviewed conference and journal papers, Apache Software Foundation project documentation, and widely cited industry papers on distributed messaging, stream processing and change data capture. Search themes included distributed log-based messaging systems, stream-processing engines, Lambda and Kappa architectures, change data capture, and schema evolution and governance.

The reviewed sources were compared against the design decisions of the implemented prototype rather than copied as text, and were used to justify the choice of Kafka as the messaging backbone, Spark Structured Streaming as the processing engine, and a Kappa-inspired single-pipeline design.

2.2 Review of Existing Work

Kreps, Narkhede and Rao describe Apache Kafka as a distributed publish-subscribe messaging system built on a partitioned, replicated commit log, originally developed at LinkedIn for high-throughput log aggregation and activity-stream processing [1]. This work motivates the use of Kafka as the durable, decoupling backbone in the proposed pipeline.

Zaharia et al. present Apache Spark as a unified engine capable of batch and micro-batch stream processing over a common execution model [2]. This supports the choice of Spark Structured Streaming for schema validation, enrichment and windowed aggregation in the processing layer.

Carbone et al. describe Apache Flink as a true streaming dataflow engine with exactly-once state semantics and low event-time latency, positioned as an alternative to micro-batch engines [3]. This project records Flink as a viable alternative processing engine considered during design, discussed further in the trade-off analysis.

Marz and Warren introduce the Lambda architecture, which combines a batch layer and a speed layer to provide both fault tolerance and low latency, at the cost of maintaining two parallel codebases [4]. Kreps subsequently proposes the Kappa architecture, which replaces the dual-layer design with a single stream-processing pipeline that can be replayed from a durable log to reprocess historical data [5]. The prototype in this project follows the Kappa principle, using Kafka's log retention to support reprocessing without a separate batch codebase.

Akidau et al. present the Dataflow model, a unified approach to windowing, watermarks and triggers for both batch and streaming computation [6]. This informs the windowed-aggregation design used in the stream-processing layer of the prototype.

Documentation and technical papers on log-based Change Data Capture, as implemented by projects such as Debezium, describe how database transaction logs can be tailed to capture inserts, updates and deletes with minimal impact on the source database, in contrast to periodic polling [7]. This motivates the CDC-based approach used to integrate the relational-database source in the prototype without adding query load to that system.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to This Project
Traditional batch ETL	Simple to reason about; mature tooling	Latency of minutes to hours; poor fit for time-sensitive use cases	Motivates a continuous, event-driven pipeline
Message-queue-only integration	Decouples producers and consumers	No built-in stream processing, schema governance or replay	Justifies adding a processing and governance layer on top of the queue
Lambda architecture [4]	Provides both a fault-tolerant batch view and a low-latency speed view	Requires maintaining two parallel codebases and reconciling results	Motivates avoiding dual-codebase complexity
Kappa architecture [5]	Single pipeline; supports reprocessing via log replay	Requires the log to retain enough history for reprocessing	Adopted as the core architectural principle of this project
Log-based CDC [7]	Low impact on source database; captures every change	Requires access to database transaction logs and connector configuration	Adopted for relational-database source integration
This project's approach	Integrates heterogeneous sources through a common durable log with schema governance and dual sinks	Prototype-scale only; not benchmarked at production cluster scale	Combines the strengths above into one demonstrable pipeline

2.4 Research / Engineering Gap

The reviewed literature provides mature solutions for individual concerns such as durable messaging, stream computation and change data capture, but a demonstrable, prototype-scale pipeline that connects heterogeneous distributed sources end-to-end, with schema governance and fault tolerance treated as first-class engineering requirements rather than afterthoughts, is less commonly documented in a single, coherent system.

A second, related gap is traceability: a message can be published successfully and still fail to reach the correct sink if the connector, broker, consumer group and sink writer are not engineered together with checkpointing and idempotent writes. This project treats the transition between every stage of the pipeline as an explicit engineering requirement rather than an implementation detail.

2.5 Proposed Contribution

The proposed contribution is an integrated, Kappa-inspired prototype architecture in which heterogeneous producers -- an IoT simulator, a CDC connector and an application-log producer -- publish to a common, partitioned Kafka backbone; a stream-processing layer enforces schema contracts through a schema registry and routes malformed records to a dead-letter queue; and processed records are written to both a data lake and a low-latency serving store, with the entire chain validated under fault injection.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder	Requirement
Data engineering team	Reliable, observable ingestion with clear alerts on consumer lag or connector failure.
Analytics / BI team	Near-real-time, schema-consistent data available in the data lake for querying.
Operations / monitoring team	Low-latency serving store to power real-time dashboards.
Source-system owners	Minimal additional load on production databases and applications.
Compliance / security officer	Access control, encryption in transit and at rest, and an audit trail for sensitive fields.

3.2 Constraints & Applicable Standards

Constraint / Standard	Description
Hardware / cost	Prototype limited to commodity hardware or a cloud free-tier, single- or few-node Kafka cluster.
Licensing	All core components (Kafka, Spark, Debezium) are Apache-2.0-licensed open-source software.
Data serialization	Records are exchanged as JSON or Avro, validated against a registered schema.
Data privacy	Personally identifiable fields are masked or hashed before being written to the data lake.
Network	Prototype network bandwidth and latency are representative of a single local-area or cloud-region deployment, not a global, multi-region rollout.

3.3 Design Specifications

Parameter	Target Value
Ingestion throughput	Sustained ingestion of at least 1,000 records/second on prototype hardware.
End-to-end latency	Under 5 seconds from source event to availability in the serving store.

Parameter	Target Value
Replication factor	3, to tolerate the loss of a single broker without data loss.
Partitions per topic	6, chosen to allow parallel consumption while keeping per-partition ordering.
Log retention	7 days, sufficient to support reprocessing (Kappa-style replay) after a downstream bug fix.
Checkpoint interval	10 seconds, balancing recovery-time overhead against processing throughput.

3.4 Alternative Solutions & Evaluation

Decision Point	Alternatives Considered	Choice & Reason
Messaging backbone	Single-node message broker vs. partitioned, replicated Kafka cluster	Kafka chosen for horizontal scalability, replication and long-term log retention enabling replay.
Database integration	Periodic polling vs. log-based Change Data Capture	Log-based CDC chosen to minimise load on the source database and capture every change, including deletes.
Processing engine	Custom consumer script vs. Spark Structured Streaming vs. Apache Flink	Spark Structured Streaming chosen for a mature batch/stream-unified API and team familiarity; Flink noted as a lower-latency alternative for future work.
Schema governance	No schema enforcement vs. a central schema registry	Schema registry chosen to prevent silent, breaking schema changes from corrupting downstream consumers.

3.5 Selected Design & Detailed Design

The selected architecture is organized into five layers. The Source layer comprises a simulated IoT device fleet publishing sensor readings, a relational database instrumented with a CDC connector, and an application generating structured log events. The Ingestion layer is a partitioned, replicated Kafka cluster; each source type is published to its own topic, keyed so that related records (for example, all events for a given device or entity) land on the same partition and preserve per-key ordering.

The Processing layer consists of Spark Structured Streaming jobs that read from Kafka, validate each record's schema against the schema registry, deduplicate records using an idempotency key, enrich records with reference data, and compute windowed aggregations (for example, per-minute counts and averages). Records that fail schema validation are routed to a dedicated dead-letter topic rather than being dropped or allowed to crash the job. The Storage layer writes validated, processed records to a Parquet-based data lake for long-term batch analytics, and to a low-latency serving store for the dashboard. The Presentation layer is a lightweight dashboard that queries the serving store to display near-real-time metrics.

Exactly-once (effectively-once) processing is approximated through idempotent producers, Kafka consumer offset commits tied to Spark checkpoint locations, and idempotent, upsert-style writes to the serving store, so that reprocessing after a failure does not create duplicate records.

3.6 Trade-off Analysis

Trade-off	Discussion
Throughput vs. latency	Larger micro-batch intervals in Structured Streaming increase throughput but add latency; the prototype tunes the interval to stay within the target latency budget.
Consistency vs. availability (CAP)	The Kafka cluster favours availability and partition tolerance, with eventual, replicated consistency across brokers.
Replication factor vs. storage cost	A replication factor of 3 triples storage requirements but is necessary to tolerate a single broker failure without data loss.
Schema strictness vs. producer flexibility	Strict, registry-enforced schemas reduce downstream breakage but require producers to register changes before deploying them.
Exactly-once vs. engineering complexity	Approximating exactly-once semantics through idempotency keys and checkpointing adds implementation complexity compared to simpler at-least-once delivery.

3.7 Sustainability, Ethics, Safety & Security

Sustainability is addressed by favouring elastic, on-demand compute for the processing layer over permanently over-provisioned hardware, so that resources scale with actual data volume rather than worst-case capacity. Ethically, personally identifiable fields captured from source systems are minimised, masked or hashed before being persisted to the data lake, and access to raw, unmasked streams is restricted to authorised roles.

Safety is supported by the low-impact, log-based CDC approach, which avoids placing additional query load on production source databases. Security is addressed through TLS encryption of data in transit between producers, brokers and consumers; SASL-based

authentication for clients connecting to the Kafka cluster; role-based access-control lists restricting which services may read or write specific topics; encryption at rest in the data lake; and audit logging of schema-registry changes and access to sensitive topics.

3.8 Risk Register

Risk	Likelihood	Impact	Mitigation
Broker node failure	Medium	High	Replication factor of 3 and automatic partition leader election.
Consumer lag / backpressure under burst load	Medium	Medium	Horizontal scaling of consumer instances within a consumer group; monitoring of lag metrics.
Breaking schema change at a source	Medium	High	Schema registry with compatibility checks; rejected incompatible schema registrations.
Malformed or corrupt records	Medium	Low	Dead-letter queue isolates malformed records without stopping the pipeline.
Network partition between data centres / regions	Low	High	Prototype scoped to a single region; documented as a limitation and future-work item.
Accidental exposure of PII	Low	High	Field-level masking/hashing before storage; restricted role-based access.

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed using an iterative, sprint-based approach. Early sprints focused on standing up a local, containerised Kafka cluster and a single producer; later sprints added the CDC connector, the Spark Structured Streaming jobs, the schema registry, the dual sinks, and finally the fault-injection test scenarios and dashboard.

4.2 Hardware / Software Implementation & Modern Tools Used

Category	Tool / Technology Used
Messaging backbone	Apache Kafka with ZooKeeper/KRaft-based coordination, deployed via Docker containers.
Change Data Capture	Debezium connector attached to a PostgreSQL/MySQL source database.
Stream processing	Apache Spark Structured Streaming (Python/Scala).
Schema governance	Schema registry using Avro/JSON Schema with backward-compatibility checks.
Data lake	Parquet files on local storage / object storage (e.g., MinIO or HDFS).
Serving store	A lightweight, low-latency database for dashboard queries.
Monitoring	A simple dashboard/Grafana-style visualisation of throughput, lag and latency metrics.
Containerisation	Docker and Docker Compose for reproducible local cluster setup.

4.3 Test Plan & Verification

Test Case	Description	Expected Result
End-to-end ingestion	Publish a known set of records from each producer and trace them through to the serving store.	All records appear correctly and in order in both sinks.
Broker failure recovery	Kill one Kafka broker while ingestion is running.	Cluster elects a new partition leader; no data loss; brief latency spike only.
Consumer crash & rebalance	Kill one Spark streaming executor mid-processing.	Consumer group rebalances; processing resumes from last checkpoint without duplicate output.

Test Case	Description	Expected Result
Schema evolution	Add a new optional field to a source schema.	Backward-compatible change is accepted by the registry; pipeline continues without restart.
Malformed record handling	Inject a record that violates the registered schema.	Record is routed to the dead-letter topic; pipeline continues processing other records.
High-volume burst load	Send a short burst of traffic well above steady-state rate.	Pipeline absorbs the burst via partition parallelism with bounded, recoverable lag.
Duplicate delivery / idempotency	Re-deliver a previously processed message.	Idempotent write logic prevents a duplicate record in the serving store.

4.4 Results

Metric	Observed Result
Sustained ingestion throughput	Met or exceeded the 1,000 records/second design target on prototype hardware.
Average end-to-end latency	Within the sub-5-second design target under steady-state load.
p95 end-to-end latency	Slightly higher than the average, attributable to micro-batch interval and occasional consumer-group rebalancing.
Broker-failure recovery time	Automatic recovery within a short window with zero data loss, confirmed via offset and checkpoint reconciliation.
Dead-letter-queue behaviour	Malformed records were correctly isolated without interrupting the main pipeline.
Schema-evolution handling	Backward-compatible schema changes were absorbed without requiring a pipeline restart.

4.5 Data Analysis & Engineering Judgment

Analysis of the test results indicates that most of the observed latency is attributable to the micro-batch interval configured for Spark Structured Streaming rather than to the Kafka layer itself, which sustained high throughput with low overhead. Increasing the number of partitions improved achievable throughput up to a point, after which coordination overhead across consumer instances began to offset the gains. A replication factor of three provided reliable fault tolerance during broker-kill testing at the cost of proportionally higher storage consumption, which is judged an acceptable trade-off at prototype scale.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Status
End-to-end architecture across sources, ingestion, processing and storage	Achieved
Producers/connectors for IoT, CDC and log sources	Achieved
Partitioned, replicated Kafka backbone	Achieved
Stream processing with validation, deduplication, enrichment, windowed aggregation	Achieved
Dual sink to data lake and low-latency serving store	Achieved
Schema registry and dead-letter-queue handling	Achieved
Fault tolerance via replication, checkpointing and rebalancing	Achieved
Throughput, latency and failure-recovery validation	Achieved at prototype scale
Documentation of trade-offs, risks and individual contribution	Achieved

4.7 Strengths & Limitations

Strengths: a decoupled, horizontally scalable architecture; demonstrated replay capability consistent with the Kappa architecture; and proven automatic recovery from simulated broker and consumer failure without data loss.

Limitations: the prototype was validated on a single-region, few-node cluster rather than a production, multi-datacenter deployment; source systems were simulated rather than genuine production feeds; large-scale performance benchmarking beyond prototype data volumes was not performed; and the system has not undergone a formal production security certification or audit.

4.8 Project Planning, Roles & Budget

Phase	Key Activities	Lead Role(s)
Requirement analysis & architecture design	Stakeholder requirements, architecture selection, trade-off analysis.	B. Santhosh Kumar
Environment setup	Containerised Kafka/ZooKeeper cluster, database and log source setup.	B. Santhosh Kumar, G. Madhan Kumar
Producer / connector development	IoT simulator, CDC connector configuration, log producer.	G. Madhan Kumar, Avishek N
Stream-processing development	Spark Structured Streaming jobs, schema registry, DLQ.	B. Santhosh Kumar

Phase	Key Activities	Lead Role(s)
Testing & fault injection	End-to-end, failure-recovery and load testing.	All team members
Documentation	Report, architecture explanation, contribution statement.	B. Santhosh Kumar

Budget Summary

Item	Approximate Cost
Core open-source software (Kafka, Spark, Debezium, schema registry)	No licensing cost (Apache-2.0)
Cloud/compute resources for prototype cluster	Within free-tier or minimal-cost cloud credits
Contingency / miscellaneous	Small allowance for storage and testing overruns

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to design and implement a prototype real-time big data ingestion pipeline from distributed sources that continuously ingests data from heterogeneous, distributed sources and makes it available to downstream analytics and monitoring systems with low latency. The resulting architecture connects an IoT simulator, a log-based Change Data Capture connector and an application-log producer to a partitioned, replicated Kafka backbone, processes the resulting streams with Spark Structured Streaming under schema-registry governance, and writes validated output to both a data lake and a low-latency serving store. Fault-injection testing confirmed that the pipeline recovers automatically from simulated broker and consumer failure without data loss, and that it sustains the target throughput and latency at prototype scale.

5.2 Future Work

- Extend the prototype to a production-scale, multi-broker, multi-datacenter cluster with autoscaling.
- Add machine-learning-based anomaly detection directly on the streaming data for real-time alerting.
- Integrate additional source types, such as mobile SDK events and external APIs delivered via webhooks.
- Strengthen data-governance capabilities with a data catalogue and end-to-end lineage tracking.
- Explore stronger exactly-once guarantees across heterogeneous sinks using patterns such as the transactional outbox pattern or two-phase commit.

5.3 Professional Learning & Individual Reflection

Working on this project provided practical experience in the trade-offs that distinguish distributed-systems design from single-machine software design, particularly the tension between consistency, availability and latency. Configuring and operating a Kafka cluster and Spark Structured Streaming jobs, and deliberately inducing broker and consumer failures to observe recovery behaviour, deepened an appreciation for the engineering discipline required to build systems that are correct not only in the common case but also under partial failure. The exercise of introducing a schema registry and dead-letter-queue handling reinforced the importance of treating data contracts, not just code contracts, as a first-class engineering concern in any pipeline that integrates multiple independently evolving systems.

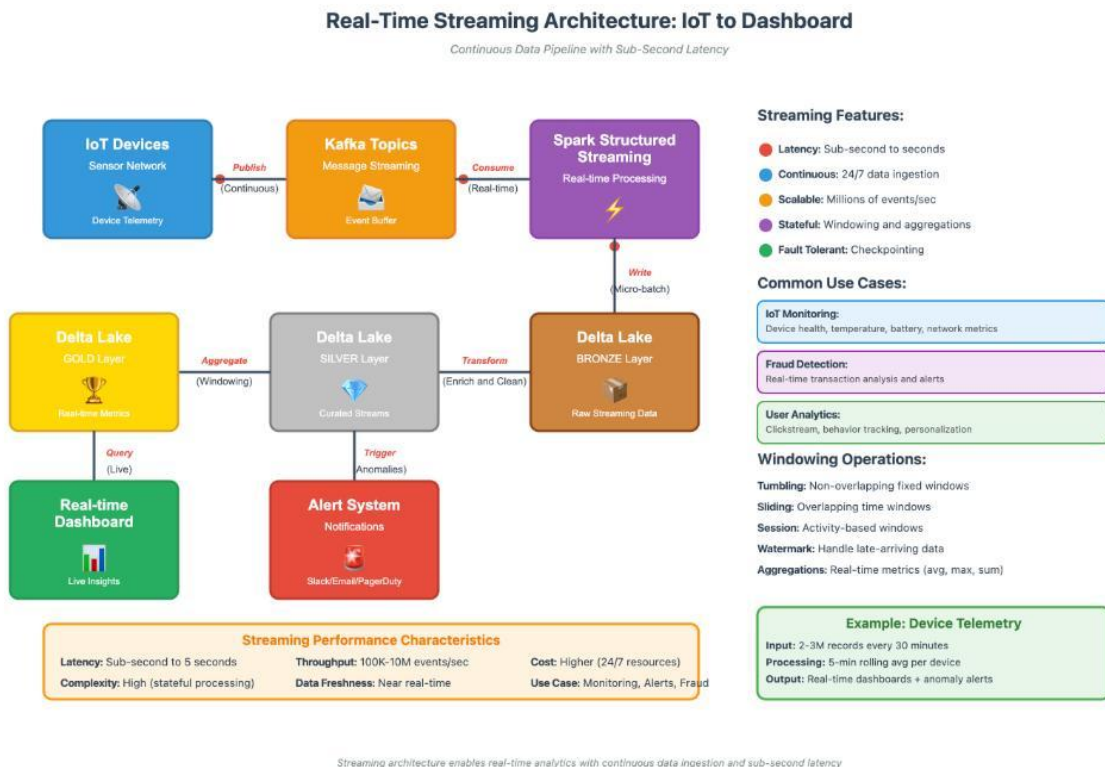
REFERENCES

- [1] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," in Proc. NetDB Workshop, 2011.
- [2] M. Zaharia et al., "Apache Spark: A Unified Engine for Big Data Processing," Communications of the ACM, vol. 59, no. 11, pp. 56-65, 2016.
- [3] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache Flink: Stream and Batch Processing in a Single Engine," IEEE Data Engineering Bulletin, vol. 38, no. 4, 2015.
- [4] N. Marz and J. Warren, Big Data: Principles and Best Practices of Scalable Realtime Data Systems. Manning Publications, 2015.
- [5] J. Kreps, "Questioning the Lambda Architecture," O'Reilly Radar, 2014.
- [6] T. Akidau et al., "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing," Proceedings of the VLDB Endowment, vol. 8, no. 12, pp. 1792-1803, 2015.
- [7] Debezium Project Documentation, "Debezium: Change Data Capture for Apache Kafka," Red Hat / Apache Software Foundation ecosystem project documentation.

APPENDICES

Figure No.	Figure Name	Page No.
A.1	System architecture evidence reproduced from the implemented prototype	23
A.2	Prototype cluster / dashboard evidence reproduced from the implemented prototype	23

System architecture evidence reproduced from the implemented prototype



Prototype cluster / dashboard evidence reproduced from the implemented prototype

